
University of Michigan–Ann Arbor

Department of Electrical Engineering and Computer Science

EECS 475 Introduction to Cryptography, Fall 2024

Instructor: Ke Wu

GSI: Forest Zhang

Homework 2

Student: YOUR NAME HERE (username here)

Collaborators (if any): Name/Exercise Number(s), Name/Exercise Numbers(s)

Please refer to the course information document for important collaboration policies. Among other policies, you must list all collaborators and, moreover, cite any external resources that you have used for each exercise. You must write all solutions on your own and may not share this material outside class.

Due: September 26, 2024 (at 11:59 PM US Eastern Time)

Exercise 1

(15 points) Recall the following definitions:

- A non-negative function $f : \mathbb{N} \rightarrow \mathbb{R}$ is *polynomially bounded*, written $f(n) = \text{poly}(n)$, if $f(n) = O(n^c)$ for some constant $c \geq 0$.
- A non-negative function $\varepsilon : \mathbb{N} \rightarrow \mathbb{R}$ is *negligible*, written $\varepsilon(n) = \text{negl}(n)$, if it decreases faster than the inverse of any polynomial. Formally: $\lim_{n \rightarrow \infty} \varepsilon(n) \cdot n^c = 0$ for any constant $c \geq 0$. (Otherwise, we say that $\varepsilon(n)$ is *non-negligible*.)

Are following functions necessarily negligible? Prove your answer. (Think: why doesn't the base of the logarithm matter in both cases?)

(a) $\varepsilon_1(n) = \frac{1}{\log(n^{\log n})}$

(b) $\varepsilon_2(n) = \frac{1}{2^{(\log^2 n)}}$

(c) $\varepsilon_3(n) = v(n)^{1/50}$, where $v(n)$ is a negligible function.

Solution

- (a) **(5 points)** Here $\varepsilon_1(n)$ is non-negligible, regardless of the (constant) base of the logarithm, because by the rule $\log(a^b) = b \log a$, we have

$$\varepsilon_1(n) = \frac{1}{(\log n)^2}.$$

For any (although we only need one) positive constant c the limit $\lim_{n \rightarrow \infty} \frac{n^c}{(\log n)^2} = \infty$. So $\varepsilon_1(n)$ is non-negligible.

- (b) **(5 points)** $\varepsilon_2(n)$ is negligible, regardless of the (constant) base b of the lograithm. We can write

$$\varepsilon_2(n) = \frac{1}{2^{\log_b^2 n}} = \frac{1}{(2^{\log_b n})^{\log_b n}} = \frac{1}{(n^{\log_n 2})^{\log_b n \cdot \log_b n}} = \frac{1}{n^{\log_b 2 \cdot \log_b n}}.$$

For any positive constant c , there is some n such that $\log_b 2 \cdot \log_b n > 2c$, e.g. take any $n > 2^{2c(\log b)^2/\log 2}$. Thus

$$\lim_{n \rightarrow \infty} n^c \cdot \varepsilon_2(n) = \lim_{n \rightarrow \infty} \frac{n^c}{n^{(\log_b 2 \cdot \log_b n)}} \leq \lim_{n \rightarrow \infty} \frac{n^c}{n^{2c}} = 0.$$

Therefore, $\varepsilon_2(n)$ is negligible.

- (c) **(5 points)** $\varepsilon_3(n)$ is negligible. Since $v(n)$ is negligible; $v(n) = o(n^{-c})$ for all $c > 0$, we know that for every $k > 0$; $\exists n_0$ s.t. $v(m) < k \cdot m^c$ for all $m > n_0$. Since exponentiation with $1/50$ is monotonically increasing, it follows that

$$v(m)^{1/50} < k^{1/50} \cdot m^{c/50} = \tilde{k} \cdot m^{\tilde{c}}$$

for $\tilde{k} = k^{1/50}$ and $\tilde{c} = c/50$.

We just showed that for all $\tilde{c} > 0$, and every $\tilde{k} > 0$, that there is a $n_0 \geq 1$ s.t. $e_6(m) < \tilde{k} \cdot m^{\tilde{c}}$ for all $m > n_0$ (take the corresponding n_0 which satisfies the inequality in the case of $v(n)$). Therefore, $\varepsilon_6(n) = o(n^{-\tilde{c}})$ for all $\tilde{c} > 0$.

Exercise 2

(15 points) Recall the definition of multi-message EAV security and multi-message CPA security. Consider the following experiments parametrized with $b \in \{0, 1\}$:

Experiment $\text{Expt}_{\text{Meav}}^{b, \mathcal{A}}(1^n)$

- $(m_{0,1}, \dots, m_{0,t}), (m_{1,1}, \dots, m_{1,t}) \leftarrow \mathcal{A}(1^n)$ where $\forall i \in [t], |m_{0,i}| = |m_{1,i}|$.
(Here the adversary is choosing two vectors of messages for the challenger to encrypt.)
- $k \xleftarrow{\$} \text{Gen}(1^n)$ and $\forall i \in [t], c_i \leftarrow \text{Enc}_k(m_{b,i})$.
(The challenger chooses one of the two vectors depending on the parameter b to encrypt and returns it to the adversary.)
- $b' \leftarrow \mathcal{A}(c_1, \dots, c_t)$.
(The adversary guesses which output it sees)
- Output b' .

We say that $(\text{Gen}, \text{Enc}, \text{Dec})$ is multi-message EAV-secure if and only if for any PPT $\mathcal{A}$,

$$\{\text{Expt}_{\text{Meav}}^{0, \mathcal{A}}(1^n)\}_n \approx \{\text{Expt}_{\text{Meav}}^{1, \mathcal{A}}(1^n)\}_n.$$

Experiment $\text{Expt}_{\text{LRcpa}}^{b, \mathcal{A}}(1^n)$

- $k \xleftarrow{\$} \text{Gen}(1^n)$.
- $b' \leftarrow \mathcal{A}^{\text{LR}_k^b(\cdot, \cdot)}(1^n)$ where $\text{LR}_k^b(m_0, m_1)$ outputs $\text{Enc}_k(m_b)$.
(The adversary gets oracle access to the LR_k^b oracle which will always return the left message or right message depending on the parameter b when given a message pair. The adversary can make $\text{poly}(n)$ queries.)
- Output b' .

We say that $(\text{Gen}, \text{Enc}, \text{Dec})$ is multi-message CPA-secure if and only if for any PPT $\mathcal{A}$,

$$\{\text{Expt}_{\text{LRcpa}}^{0, \mathcal{A}}(1^n)\}_n \approx \{\text{Expt}_{\text{LRcpa}}^{1, \mathcal{A}}(1^n)\}_n.$$

Prove that Multi-message CPA security implies multi-message EAV security using the distinguisher version of definitions above.

Solution

We must show that for all encryption schemes Π , if Π is multi-message CPA secure, then it is multi-message EAV secure. To do so, we prove the contrapositive: We will show that if there exists a scheme Π that is *not* multi-message EAV secure then it is *not* multi-message CPA secure.

Indeed, assume there exists an adversary $\mathcal{A}$ that breaks the multi-message EAV security of Π with non-negligible probability $\epsilon(n)$. That is

$$|\Pr[\text{Expt}_{\text{mmeav}}^{1,\mathcal{A}} = 1] - \Pr[\text{Expt}_{\text{mmeav}}^{0,\mathcal{A}} = 1]| = \epsilon(n).$$

Then we can construction adversary $\mathcal{B}$ that breaks the multi-message CPA security of Π with probability $\epsilon(n)$ using $\mathcal{A}$. That is, we want to show that the following probability is $\epsilon(n)$.

$$|\Pr[\text{Expt}_{\text{LRcpa}}^{1,\mathcal{B}} = 1] - \Pr[\text{Expt}_{\text{LRcpa}}^{0,\mathcal{B}} = 1]| \quad (1)$$

We construct such an adversary $\mathcal{B}$ as follows.

Adversary $\mathcal{B}^{\text{LR}_k^b}(1^n)$:

1. Run $\mathcal{A}$ to get it's attack messages:

$$(m_{0,1}, \dots, m_{0,t}), (m_{1,0}, \dots, m_{1,t}) \leftarrow \mathcal{A}(1^n)$$

2. For all $i \in \{1, \dots, t\}$, query the provided oracle with messages $(m_{0,i}, m_{1,i})$:

$$c_{b,i} \leftarrow \text{LR}_k^b(m_{0,i}, m_{1,i}).$$

3. Feed the resulting ciphertexts back to $\mathcal{A}$ to get it's guess b :

$$b \leftarrow \mathcal{A}(c_{b,1}, \dots, c_{b,t})$$

4. Output b .

We now argue that $\mathcal{B}$'s success probability is $\epsilon(n)$. By definition of $\text{Expt}_{\text{LRcpa}}$, we have that Equation (1) is equivalent to

$$|\Pr[\mathcal{B}^{\text{LR}_k^1}(1^n) = 1] - \Pr[\mathcal{B}^{\text{LR}_k^0}(1^n) = 1]|. \quad (2)$$

By definition of $\mathcal{B}$, we have that Equation (2) is equivalent to

$$\left| \Pr[\mathcal{A}(c_{b,0}, \dots, c_{b,n}) = 1 | c_{b,i} \leftarrow \text{LR}_k^1(m_{0,i}, m_{1,i}), (m_{0,1}, \dots, m_{0,t}), (m_{1,0}, \dots, m_{1,t}) \leftarrow \mathcal{A}(1^n)] - \Pr[\mathcal{A}(c_{b,0}, \dots, c_{b,n}) = 1 | c_{b,i} \leftarrow \text{LR}_k^0(m_{0,i}, m_{1,i}), (m_{0,1}, \dots, m_{0,t}), (m_{1,0}, \dots, m_{1,t}) \leftarrow \mathcal{A}(1^n)] \right| \quad (3)$$

By definition of LR, we have that Equation (3) is equivalent to

$$\left| \Pr[\mathcal{A}(c_{b,0}, \dots, c_{b,n}) = 1 | c_{b,i} \leftarrow \text{Enc}_k(m_{1,i}), (m_{0,1}, \dots, m_{0,t}), (m_{0,1}, \dots, m_{0,t}) \leftarrow \mathcal{A}(1^n)] - \Pr[\mathcal{A}(c_{b,0}, \dots, c_{b,n}) = 1 | c_{b,i} \leftarrow \text{Enc}_k(m_{0,i}), (m_{0,1}, \dots, m_{0,t}), (m_{0,1}, \dots, m_{0,t}) \leftarrow \mathcal{A}(1^n)] \right| \quad (4)$$

But this is precisely the definition of $\text{Expt}_{\text{mmeav}}^{b, \mathcal{A}}$. That is, Equation (4) is equivalent to

$$\left| \Pr[\text{Expt}_{\text{mmeav}}^{1, \mathcal{A}} = 1] - \Pr[\text{Expt}_{\text{mmeav}}^{0, \mathcal{A}} = 1] \right|,$$

which, by assumption, is equivalent to $\epsilon(n)$. Thus, following the chain of equalities we have that

$$\left| \Pr[\text{Expt}_{\text{LRcpa}}^{1, \mathcal{B}} = 1] - \Pr[\text{Expt}_{\text{LRcpa}}^{0, \mathcal{B}} = 1] \right| = \epsilon(n). \quad (5)$$

Exercise 3

(20 points) Suppose that $G : \{0, 1\}^n \rightarrow \{0, 1\}^{2n}$ is a secure pseudorandom generator that expands an n -bit random seed into a $2n$ -bit pseudorandom string. For each of the following questions: first, give a YES/NO answer; if the answer is YES, please prove it; if the answer is NO, please provide a counter-example.

- (a) Let $\wedge$ denote the bit-wise *and* operation. Is $G((0^{n-1}||1) \wedge s)$ a secure pseudorandom generator that expands a random seed of length n into a pseudorandom string of length $2n$?
- (b) Is $G(\text{first half of } G(s)) || G(\text{second half of } G(s))$ a secure pseudorandom generator that maps n bits to $4n$ bits?

Solution

- (a) This is not a PRG. Let $G'(s) = G((0^{n-1}||1) \wedge s)$. For any s , there are only two possibilities of the output of $G(\cdot)$: $a = G(0^n)$ and $b = G(0^{n-1}||1)$. Consider the following distinguisher $\mathcal{D}$:

Upon receiving a string x of length $2n$, $\mathcal{D}$ checks whether it equals a or b . If yes, outputs 1. Otherwise, outputs 0.

Then the probability of $\mathcal{D}$ distinguishing $G'(\cdot)$ and U_{2n} is

$$\left| \Pr[\mathcal{D} \text{ outputs } 1 \mid x \xleftarrow{\$} U_{2n}] - \Pr[\mathcal{D} \text{ outputs } 1 \mid s \xleftarrow{\$} U_n, x = G'(s)] \right| = 1 - \frac{1}{2^{2n-1}}.$$

That is, the distinguisher $\mathcal{D}$ can distinguish the output from $G'(\cdot)$ and U_{2n} with a non-negligible probability. Thus G' is not a secure PRG.

(b) Let $G'(s) = G(\text{first half of } G(s)) \parallel G(\text{second half of } G(s))$. Consider the following hybrids:

- $H_0 := G'(s), s \xleftarrow{\$} U_n$.
- $H_1 := G(u_1) \parallel G(u_2), u_1, u_2 \xleftarrow{\$} U_n$.
- $H_2 := u \parallel G(u_2), u_2 \xleftarrow{\$} U_n, u \xleftarrow{\$} U_{2n}$.
- $H_3 := u \parallel u', u, u' \xleftarrow{\$} U_{2n}$.

$H_0 \approx H_1$: Suppose for the sake of contradiction that there exists a distinguisher $\mathcal{D}$ that can distinguish H_0 from H_1 with probability at least $\frac{1}{p(n)}$ for some polynomial $p(\cdot)$. Then we can construct a distinguisher $\mathcal{D}'$ that distinguish $G(U_n)$ from U_{2n} as follows: Upon receiving a string $x \in \{0, 1\}^{2n}$, it parse $x = x_1 \parallel x_2$ such that $x_1, x_2 \in \{0, 1\}^n$. It then sends $G(x_1) \parallel G(x_2)$ to $\mathcal{D}$. It then outputs whatever $\mathcal{D}$ outputs.

Since when x is drawn from U_{2n} , $\mathcal{D}'$'s view is identical to H_1 ; when x is drawn from $G(U_n)$, $\mathcal{D}'$'s view is identical to H_0 . Thus, $\mathcal{D}'$ can distinguish $G(U_n)$ and U_{2n} with probability at least $\frac{1}{p(n)}$, which breaks the security of $G(\cdot)$.

$H_1 \approx H_2$: Suppose for the sake of contradiction that there exists a distinguisher $\mathcal{D}$ that can distinguish H_0 from H_1 with probability at least $\frac{1}{p(n)}$ for some polynomial $p(\cdot)$. Then we can construct a distinguisher $\mathcal{D}'$ that distinguish $G(U_n)$ from U_{2n} as follows: Upon receiving a string $x \in \{0, 1\}^{2n}$, it randomly sample a string $u_2 \xleftarrow{\$} U_n$ and send $x \parallel G(u_2)$ to $\mathcal{D}$. It then outputs whatever $\mathcal{D}$ outputs.

Since when x is drawn from U_{2n} , $\mathcal{D}'$'s view is identical to H_2 ; when x is drawn from $G(U_n)$, $\mathcal{D}'$'s view is identical to H_1 . Thus, $\mathcal{D}'$ can distinguish $G(U_n)$ and U_{2n} with probability at least $\frac{1}{p(n)}$, which breaks the security of $G(\cdot)$.

$H_2 \approx H_3$: follow a similar argument as for $H_1 \approx H_2$.

Note that when x and y are both uniformly random, $x \parallel y$ is not necessarily uniformly random. Imagine a random variable $x \xleftarrow{\$} U_n$ and $y = x$. Then both x and y are uniformly random but $x \parallel y$ is not.

Exercise 4

(10 points) Determine whether the following F' is a pseudorandom function. If yes, prove it; if not, describe and analyze a feasible distinguisher that has non-negligible advantage in the PRF game against F' .

$F'_k(x) := G(x) \oplus k$ where $G(s)$ is a PRG.

Solution

F' is *not* a PRF. The key observation is that in the PRF game, the *attacker chooses* the inputs x , so it actually knows what $G(x)$ is (because the code of G is public). This makes it easy to detect “non-random-looking” behavior in the outputs of F'_k .

Formally, we construct a distinguisher $\mathcal{D}$ as follows: it queries on two (arbitrary) *distinct* strings x_0 and x_1 , and receives the outputs $y_0 = \mathcal{O}(x_0)$, $y_1 = \mathcal{O}(x_1)$. It then computes $G(x_0) \oplus G(x_1)$, and compares it with $y_0 \oplus y_1$. (Note that $\mathcal{D}$ can do this because the code of G is publicly known, and it chose x_0, x_1 itself.) $\mathcal{D}$ accepts if $G(x_0) \oplus G(x_1) = y_0 \oplus y_1$, and rejects otherwise.

Clearly $\mathcal{D}$ is efficient; we now analyze its advantage. In the real world, where $\mathcal{O} = F'_k$ for some (unknown, random) k , observe that $y_0 \oplus y_1 = (G(x_0) \oplus k) \oplus (G(x_1) \oplus k) = G(x_0) \oplus G(x_1)$, so $\mathcal{D}$ will *always* accept.

But in the ideal world, where $\mathcal{O} = U$ is a random function, the outputs $y_0 = U(x_0)$, $y_1 = U(x_1)$ are uniformly random and independent by definition, hence $y_0 \oplus y_1$ is uniformly random. The string $G(x_0) \oplus G(x_1)$ that we compare against is some fixed string, so the probability that $y_0 \oplus y_1$ equals it is only $1/2^\ell$ (where ℓ is the string length). Therefore, the advantage of $\mathcal{D}$ is $1 - 1/2^\ell$, which is non-negligible.

Remark: You might have been tempted to argue that F' is a PRF, using the following reasoning: for each query x , the string $G(x)$ looks like a truly random string because G is a PRG. So, each $G(x) \oplus k$ also looks like a random string, so altogether F' looks like a random function. This reasoning is faulty, because the PRG property of G says that $G(x)$ “looks random” for a *randomly chosen, secret* seed x . But in this problem’s scenario, x is *not random*—it is *chosen* by the attacker—and hence it is also *known* to the attacker. Clearly, $G(x)$ does not look random at all to someone who knows x !

Exercise 5

(10 points) Give a distinguisher that has non-negligible advantage in the PRF game against the following F' .

$F'_{k_1||k_2}(x_1||x_2) := F_{k_1}(x_1)||F_{k_2}(x_2)$ where F is a PRF and $|x_1| = |x_2|$. Here k_1 and k_2 are drawn uniformly from the keyspace of F .

Solution

F' is *not* a PRF. The intuition is that a random function outputs a random, independent output for each distinct input, so the behavior of a PRF should “look like” that as well. But for F' , if we change just the x_2 part of the input (leaving the x_1 part unchanged), we have changed the input but the first part of the output is unchanged. This is unlikely to be the case for a random function, so we can build a distinguisher that tests for this condition.

Formally, we define a distinguisher $\mathcal{D}$ that queries its oracle $\mathcal{O}$ on two inputs $x_1||x_2$ and $x_1||x'_2$, where $|x_1| = |x_2| = |x'_2|$ and $x_2 \neq x'_2$. After receiving the respective outputs $y_1 = u_1||v_1 = \mathcal{O}(x_1||x_2)$ and $y_2 = u_2||v_2 = \mathcal{O}(x_1||x'_2)$ where $|u_1| = |u_2| = |v_1| = |v_2|$, the distinguisher accepts if $u_1 = u_2$ and rejects otherwise.

Clearly $\mathcal{D}$ is efficient. To analyze its advantage, observe that in the real world, where the oracle $\mathcal{O} = F'_{k_1,k_2}$ for some (unknown, random) k_1, k_2 , because $u_1 = F_{k_1}(x_1) = u_2$, we see that $\mathcal{D}$ *always* accepts. But in the ideal world, where the oracle $\mathcal{O}$ is a truly random function U , because the inputs $x_1||x_2 \neq x_1||x'_2$ are different, the first halves u_1, u_2 of the outputs are random and independent, and hence they are equal only with probability $1/2^\ell$, where $\ell = |u_1| > 0$. Therefore, in the ideal world $\mathcal{D}$ accepts with probability only $1/2^\ell$, so the advantage of $\mathcal{D}$ is $1 - 1/2^\ell$, which is non-negligible.

Exercise 6

(30 points) Consider two encryption schemes $\Pi_1 = (\text{Gen}_1, \text{Enc}_1, \text{Dec}_1)$ and $\Pi_2 = (\text{Gen}_2, \text{Enc}_2, \text{Dec}_2)$ over the key space $\mathcal{K}$, message space $\mathcal{M}$ and ciphertext $\mathcal{C}$. Suppose that $\mathcal{K} = \mathcal{M} = \mathcal{C}$. Now suppose that at least one of Π_1 and Π_2 is CPA secure but you do not know which one. Construct a CPA secure encryption scheme using Π_1 and Π_2 .

Please describe your construction and prove it CPA secure, assuming that at least one of the underlying encryption schemes is CPA secure. Besides the two encryption schemes Π_1 and Π_2 , please do not use other cryptographic primitives in your construction.

(Note: such a scheme is called a cryptographic “combiner”. A cryptographic combiner can be used to hedge your risk. For example, suppose that the two encryption schemes rely on different hardness assumptions. The “combiner” approach allows you to design an encryption scheme as long as one of the underlying encryption schemes remains secure.)

Solution

We construct a CPA secure scheme $\Pi = (\text{Gen}, \text{Enc}, \text{Dec})$ as follows

$$\begin{aligned}\text{Gen}(1^n) &\mapsto (\text{Gen}_1(1^n), \text{Gen}_2(1^n)) \\ \text{Enc}((k_1, k_2), m) &\mapsto \text{Enc}_2(k_2, \text{Enc}_1(k_1, m)) \\ \text{Dec}((k_1, k_2), m) &\mapsto \text{Dec}_1(k_1, \text{Dec}_2(k_2, m))\end{aligned}$$

By observation Π satisfies correctness. To show that Π is CPA-secure, we will first assume for contradiction that there exists adversary $\mathcal{A}$ that breaks the CPA security of Π with non-negligible probability $\epsilon(n)$. Then, we can use $\mathcal{A}$ to construct adversaries $\mathcal{B}_1$ and $\mathcal{B}_2$ that break the CPA security of Π_1 and Π_2 respectively. That is, we can construct $\mathcal{B}_1$ and $\mathcal{B}_2$ such that

$$\left| \Pr[\text{Expt}_{\text{cpa}}^{1, \mathcal{B}_1} = 1] - \Pr[\text{Expt}_{\text{cpa}}^{0, \mathcal{B}_1} = 1] \right| = \epsilon(n) \quad (6)$$

and

$$\left| \Pr[\text{Expt}_{\text{cpa}}^{1, \mathcal{B}_2} = 1] - \Pr[\text{Expt}_{\text{cpa}}^{0, \mathcal{B}_2} = 1] \right| = \epsilon(n). \quad (7)$$

Because at least one of Π_1 and Π_2 is CPA-secure, this leads to a contradiction. We construct $\mathcal{B}_1$ as follows. The oracle access $\text{Enc}_{b,k}(m)$ returns $\text{Enc}_b(k, m)$.

Adversary $\mathcal{B}_1^{\text{Enc}_{1,k}}(1^n)$:

1. Sample $k_2 \leftarrow \text{Gen}_2(1^n)$.
2. Run $\mathcal{A}$. When $\mathcal{A}$ queries Enc on message m , respond with $\text{Enc}_2(k_2, \text{Enc}_{1,k}(m))$.
3. Eventually $\mathcal{A}$ finishes querying and outputs messages (m_0, m_1) .
4. Query the experiment on (m_0, m_1) to get back $c_b \leftarrow \text{Enc}_{1,k}(m_b)$.
5. Respond to $\mathcal{A}$ with $\text{Enc}_2(k_2, c_b)$.
6. $\mathcal{A}$ once again is provided the opportunity to make queries. Once again, respond to query m with $\text{Enc}_2(k_2, \text{Enc}_{1,k}(m))$.
7. Eventually, $\mathcal{A}$ finished querying and outputs its guess b . Output b .

We now argue that $\mathcal{B}_1$ succeeds with probability $\epsilon(n)$. By definition of Expt_{cpa} and construction of $\mathcal{B}$, we have that Equation (6) is equivalent to

$$\left| \Pr[\mathcal{A}^{\text{Enc}_2(k_2, \text{Enc}_{1,k}(\cdot))}(c) = 1 \mid c \leftarrow \text{Enc}_2(k_2, \text{Enc}_{1,k}(m_1)), (m_0, m_1) \leftarrow \mathcal{A}^{\text{Enc}_2(k_2, \text{Enc}_{1,k}(\cdot))}] - \Pr[\mathcal{A}^{\text{Enc}_2(k_2, \text{Enc}_{1,k}(\cdot))}(c) = 1 \mid c \leftarrow \text{Enc}_2(k_2, \text{Enc}_{1,k}(m_0)), (m_0, m_1) \leftarrow \mathcal{A}^{\text{Enc}_2(k_2, \text{Enc}_{1,k}(\cdot))}] \right| \quad (8)$$

But, by definition, we have that

$$\text{Enc}_2(k_2, \text{Enc}_{1,k}(\cdot)) = \text{Enc}_{(k_2, k)}(\cdot). \quad (9)$$

Therefore, Equation (10) is equivalent to

$$\left| \Pr[\mathcal{A}^{\text{Enc}_{(k_2, k)}(\cdot)}(c) = 1 \mid c \leftarrow \text{Enc}_{(k_2, k)}(m_1), (m_0, m_1) \leftarrow \mathcal{A}^{\text{Enc}_{(k_2, k)}(\cdot)}] - \Pr[\mathcal{A}^{\text{Enc}_{(k_2, k)}(\cdot)}(c) = 1 \mid c \leftarrow \text{Enc}_{(k_2, k)}(m_0), (m_0, m_1) \leftarrow \mathcal{A}^{\text{Enc}_{(k_2, k)}(\cdot)}] \right| \quad (10)$$

Because k_2 was sampled randomly, this is precisely the definition of Expt_{cpa} with respect to adversary $\mathcal{A}$ and protocol Π . That is, Equation (10) is equivalent to

$$\left| \Pr[\text{Expt}_{\text{cpa}}^{1, \mathcal{A}} = 1] - \Pr[\text{Expt}_{\text{cpa}}^{0, \mathcal{A}} = 1] \right|,$$

which, by assumption, is equal to $\epsilon(n)$. Therefore, following the chain of equalities we have that Equation (6) holds.

Next we construct $\mathcal{B}_2$ as follows

Adversary $\mathcal{B}_2^{\text{Enc}_{2,k}}(1^n)$:

1. Sample $k_1 \leftarrow \text{Gen}_1(1^n)$.
2. Run $\mathcal{A}$. When $\mathcal{A}$ queries Enc on message m respond with $\text{Enc}_{2,k}(\text{Enc}_1(k_1, m))$.
3. Eventually $\mathcal{A}$ finishes querying and outputs messages (m_0, m_1) .
4. Query the experiment on $(\text{Enc}_1(k_1, m_0), \text{Enc}_1(k_1, m_1))$ to get back

$$c_b \leftarrow \text{Enc}_{2,k}(\text{Enc}_1(k_1, m_b)).$$

5. Respond to $\mathcal{A}$ with $\text{Enc}_2(k_2, c_b)$.
6. $\mathcal{A}$ once again is provided the opportunity to make queries. Once again, respond to query m with $\text{Enc}_{2,k}(\text{Enc}_1(k_1, m))$
7. Eventually, $\mathcal{A}$ finished querying and outputs it's guess b . Output b .

The argument for why $\mathcal{B}_2$ succeeds with probability $\epsilon(n)$ is symmetric to the argument for $\mathcal{B}_1$.